Industrial Internship Report on

1.Smart City Traffic Forecasting & 2.Crop vs Weed Detection using YOLOv8

Prepared by

Utsa Das

Executive Summary
This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT). This internship was focused on a project/problem statement provided by UCT. We had to finish the project including the report in 6 weeks' time. My project was (Tell about ur Project) This internship gave me a very good opportunity to get exposure to Industrial problems and design/implement solution for that. It was an overall great experience to have this internship.

TABLE OF CONTENTS

1 Preface

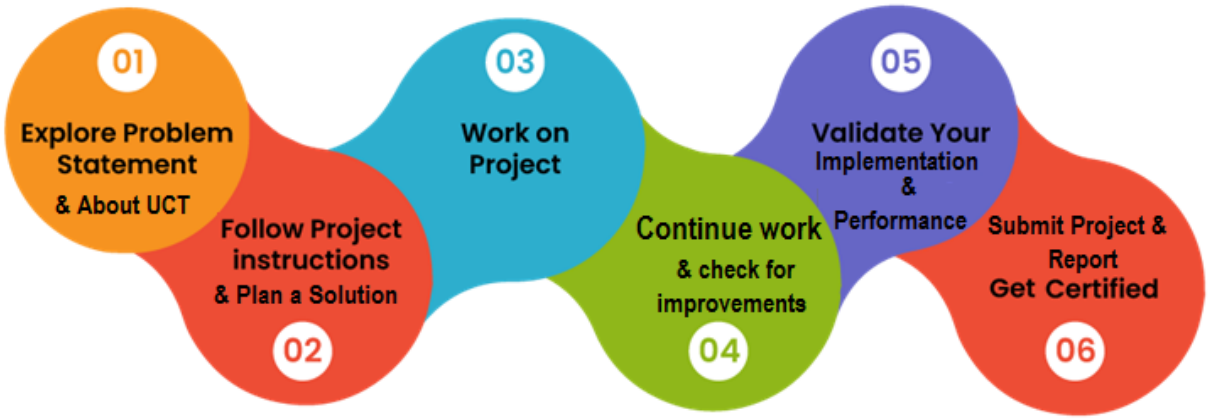
Summary of the whole 6 weeks' work.

About need of relevant Internship in career development.

Brief about Your project/problem statement.

Opportunity given by USC/UCT.

How Program was planned



Your Learnings and overall experience.

Thank to all (with names), who have helped you directly or indirectly.

Your message to your juniors and peers.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.

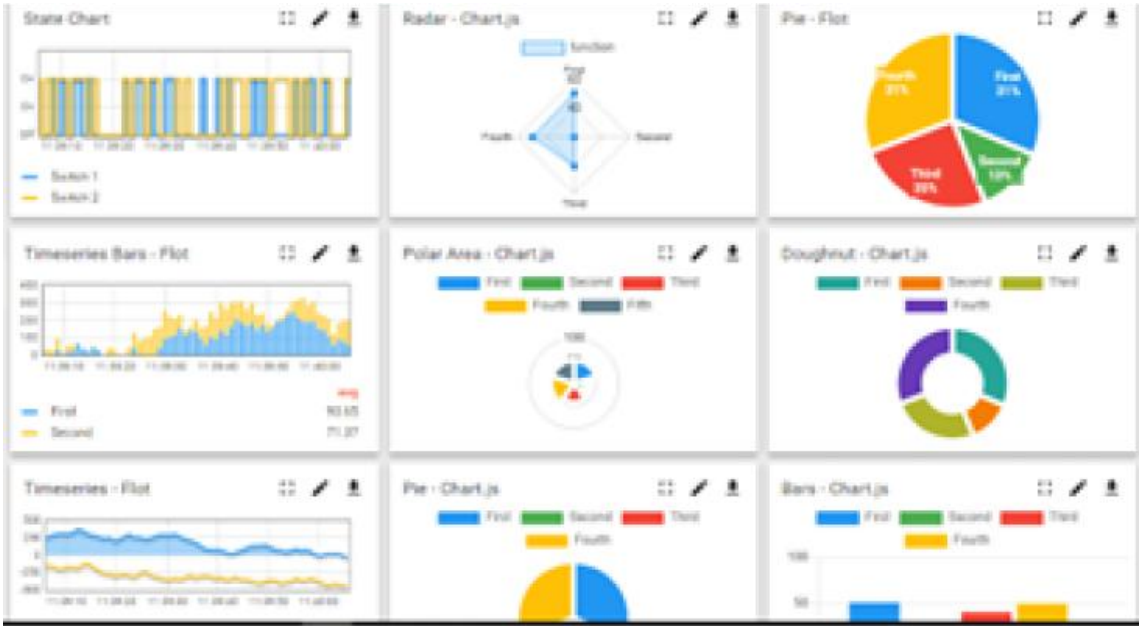
i. UCT IoT Platform (UCT Insight)

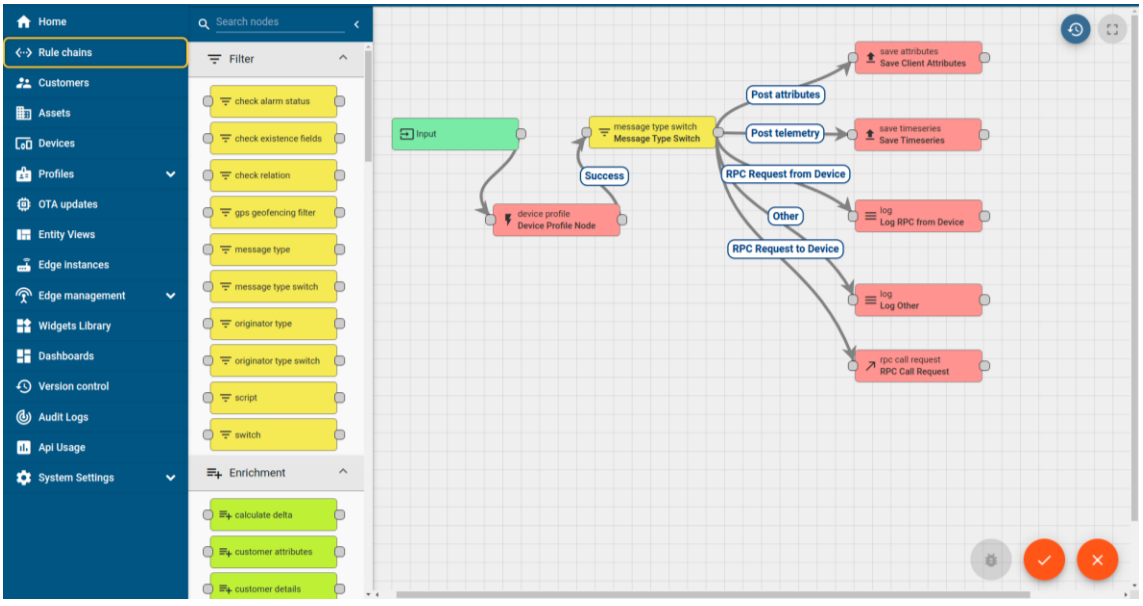
UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable "insight" for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine





ii. Smart Factory Platform (FACTORY WATCH)

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleased the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they what to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.

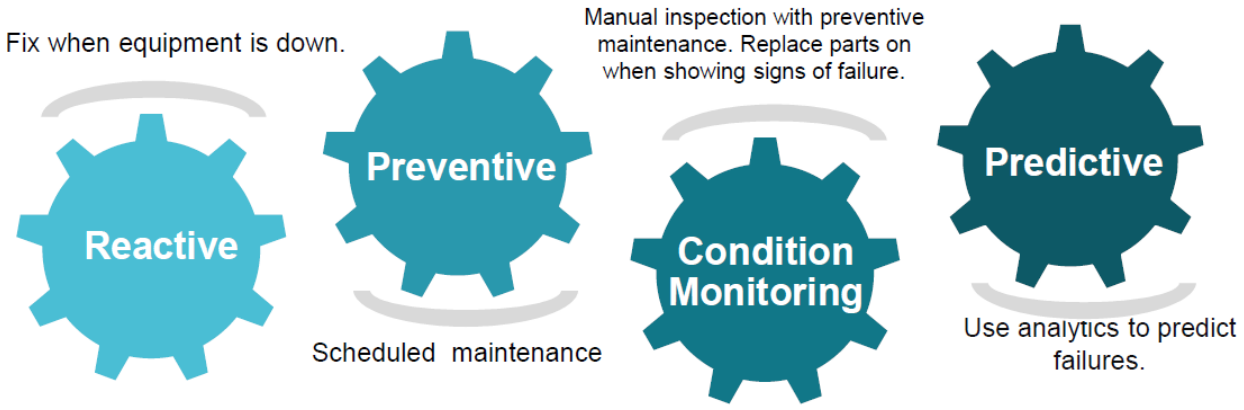
Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i

iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN teschnology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

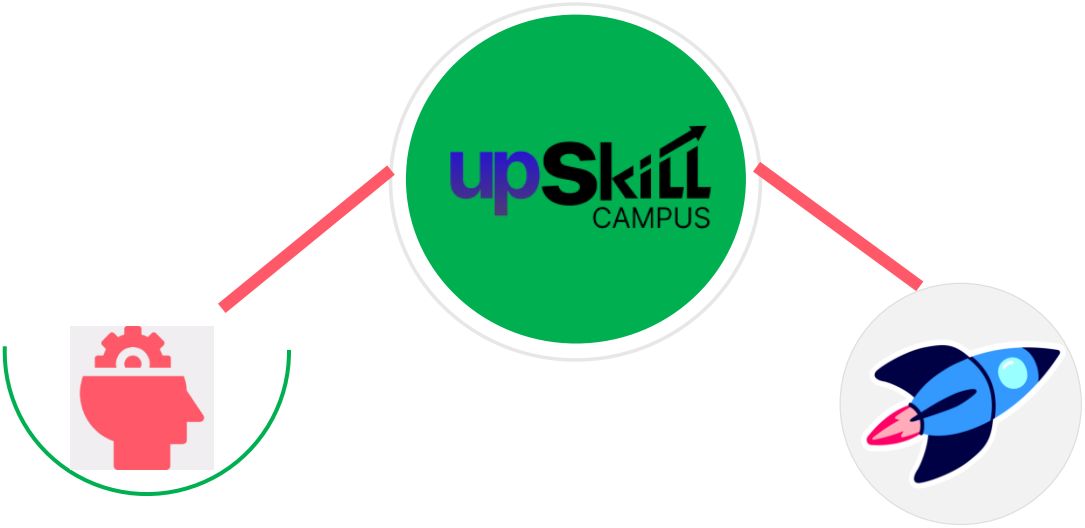
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

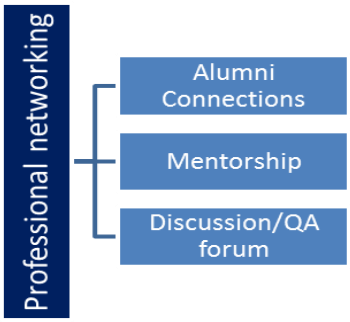
USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.

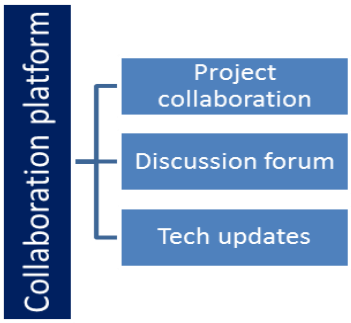


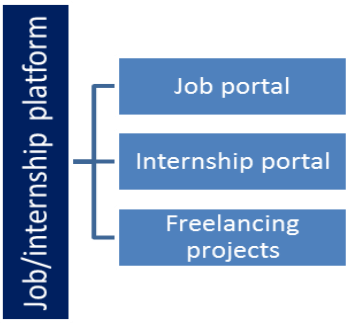
Seeing need of upskilling in self paced manner along-with additional support services e.g. Internship, projects, interaction with Industry experts, Career growth Services

upSkill Campus aiming to upskill 1 million learners in next 5 year

https://www.upskillcampus.com/







2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The objective for this internship program was to

- ☛ get practical experience of working in the industry.
- ☛ to solve real world problems.
- ☛ to have improved job prospects.
- ☛ to have Improved understanding of our field and its applications.
- ☛ to have Personal growth like better communication and problem solving.

2.5 Reference

[1] Box, G. E. P., Jenkins, G. M., Reinsel, G. C., Ljung, G. M., *Time Series Analysis: Forecasting and Control*, Wiley.

[2] Hochreiter, S., Schmidhuber, J., "Long Short-Term Memory," *Neural Computation*, 1997.

[3] Hyndman, R. J., Athanasopoulos, G., *Forecasting: Principles and Practice*, OTexts.

[4] Goodfellow, I., Bengio, Y., Courville, A., *Deep Learning*, MIT Press.

[5] Chollet, F., *Deep Learning with Python*, Manning Publications.

[6] Redmon, J. et al., "You Only Look Once: Unified, Real-Time Object Detection," *CVPR*, 2016.

[7] Ultralytics, "YOLOv8 Documentation" https://docs.ultralytics.com

2.6 Glossary

Terms	Acronym
Artificial Intelligence	AI
Machine Learning	ML
Deep Learning	DL
Long Short-Term Memory	LSTM
Gated Recurrent Unit	GRU
Autoregressive Integrated Moving Average	ARIMA
Seasonal ARIMA	SARIMA
Mean Absolute Error	MAE
Root Mean Squared Error	RMSE
Mean Average Precision	mAP
Intersection over Union	IoU
Representational State Transfer	REST
Application Programming Interface	API
Open Neural Network Exchange	ONNX

3 Problem Statement

◇ 3.1 Smart City Traffic Forecasting

- **Problem Statement**

Urban areas are experiencing rapid growth in vehicle usage, leading to **severe traffic congestion**, increased travel time, fuel wastage, and environmental pollution. Traditional traffic management systems rely on static rules and manual monitoring, which fail to adapt to dynamic traffic patterns.

The core problem is:

How to accurately predict future traffic volume using historical data to enable intelligent, real-time traffic management?

Key challenges:

- Traffic data is **time-dependent and non-linear**
- Presence of **daily, weekly, and seasonal patterns**
- Sudden fluctuations (accidents, peak hours)
- Need for **real-time prediction capability**

- ◇ **3.2 Crop vs Weed Detection using YOLOv8**
- **Problem Statement**

In agriculture, weeds compete with crops for essential resources such as water, sunlight, and nutrients, significantly reducing crop yield. Traditional weed control methods are:

- Manual (labor-intensive)
- Chemical-based (environmentally harmful)
- Inefficient (non-selective spraying)

The core problem is:

How to automatically detect and differentiate crops and weeds from field images to enable precision agriculture?

Key challenges:

- Visual similarity between crops and weeds

- Varying lighting and environmental conditions
- Real-time detection requirements
- Need for high accuracy with low latency

4. EXISTING vs PROPOSED SOLUTION

4.1 Smart City Traffic Forecasting

Existing Solutions

1. **Rule-Based Systems**
   - Fixed traffic signal timing
   - No adaptability to real-time conditions
2. **Statistical Models (ARIMA)**
   - Captures linear patterns
   - Works well for stationary data

Limitations

- Cannot handle:
  - Complex non-linear patterns
  - Long-term dependencies
  - Real-time dynamic changes
- Poor performance during peak fluctuations
- Lack of scalability and deployment readiness

Proposed Solution

A **hybrid, multi-stage intelligent forecasting system**:

1. **Baseline Model** → ARIMA
2. **Improved Model** → SARIMA (seasonality-aware)
3. **Advanced Model** → LSTM (deep learning-based)
4. **Deployment Layer** → REST API + Dashboard

- **Value Addition**

  - Transition from **basic forecasting → real-time intelligent system**
  - Integration of:
    - Deep learning (LSTM)
    - API-based deployment
    - Live dashboard visualization
  - Handles:
    - Non-linear traffic behavior
    - Seasonal patterns
    - Real-time predictions

- ## 🔹 4.2 Crop vs Weed Detection

- **Existing Solutions**

1. **Manual Detection**
   - Human inspection
   - Time-consuming and error-prone
2. **Traditional Image Processing**
   - Edge detection, thresholding
   - Limited accuracy in complex environments
3. **Basic ML Models**
   - Classification-based (not detection)
   - Cannot localize objects

- **Limitations**
  - No real-time capability
  - Poor performance in:
    - Dense vegetation
    - Variable lighting
  - Cannot detect multiple objects simultaneously

- **Proposed Solution**

A **deep learning-based object detection system using YOLOv8**

Key components:

- Real-time detection pipeline
- Bounding box localization
- Multi-object classification

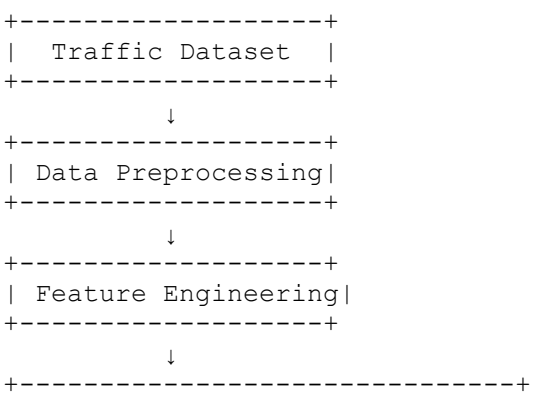
- **Value Addition**

  - High-speed detection (real-time)
  - High accuracy using deep learning
  - Deployment-ready (ONNX, TorchScript)
  - Applicable to:
    - Smart farming
    - Agricultural robotics
    - Drone-based monitoring

5. PROPOSED DESIGN / MODEL

5.1 SMART CITY TRAFFIC FORECASTING SYSTEM

5.1.1 High-Level Design




```
| Model Layer                    |
| ARIMA → SARIMA → LSTM         |
+--------------------------------+
          ↓
+--------------------+
| Prediction Output  |
+--------------------+
          ↓
+--------------------+
| REST API Layer     |
+--------------------+
          ↓
+--------------------+
| Dashboard (UI)     |
+--------------------+
```

📌 **Figure 1: High-Level Traffic Forecasting System**

- ## 🔸 5.1.2 Low-Level Design
- **Step-by-Step Flow**

1. **Input Layer**
   - Raw traffic data (DateTime, Vehicles)
2. **Preprocessing**
   - Datetime conversion
   - Missing value handling
3. **Feature Engineering**
   - Hour, Day, Month, Weekday
   - Peak-hour flags
4. **Model Layer**
   - ARIMA → baseline
   - SARIMA → seasonal patterns
   - LSTM → deep learning prediction
5. **Prediction Engine**
   - Forecast next time steps
6. **API Layer**
   - Input: recent time-series data
   - Output: predicted values (JSON)
7. **Visualization Layer**
   - Graphs (actual vs predicted)

- ### 🔸 5.1.3 Interfaces
- **Data Flow**

```
Dataset → Preprocessing → Model → API → Dashboard
```

- **Protocols Used**
  - HTTP/REST (API communication)
  - JSON (data exchange)

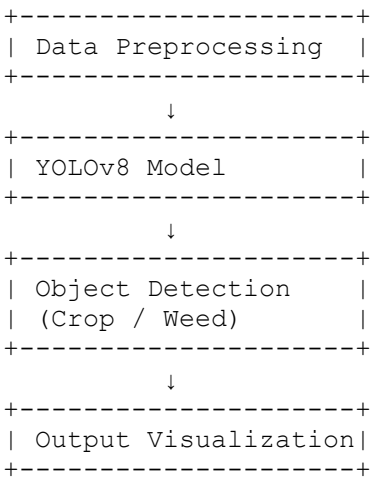
- **Flowchart**

```
Start
  ↓
Load Dataset
  ↓
Preprocess Data
  ↓
Feature Engineering
  ↓
Train Model (LSTM)
  ↓
Generate Predictions
  ↓
Send via API
  ↓
Display on Dashboard
  ↓
End
```

🔹 5.2 CROP vs WEED DETECTION SYSTEM

- ### 🔸 5.2.1 High-Level Design

```
+----------------------+
| Input Images         |
+----------------------+
          ↓
```



📌 **Figure 2: High-Level Object Detection System**

- ### 🔸 5.2.2 Low-Level Design
- **Step-by-Step Flow**

1. **Input**
   - Agricultural images
2. **Preprocessing**
   - Resize (512x512)
   - Normalization
3. **Annotation**
   - Bounding boxes (YOLO format)
4. **Model Training**
   - YOLOv8 (pretrained weights)
   - Epochs: 30
5. **Detection**
   - Bounding box prediction
   - Class label assignment
6. **Post-processing**
   - Confidence filtering
   - Non-max suppression
7. **Output**
   - Image with labeled boxes

- ◆ **5.2.3 Interfaces**
- **Data Flow**

```
Image → YOLO Model → Detection → Output Visualization
```

- **Flowchart**

```
Start
  ↓
Load Image
  ↓
Preprocess Image
  ↓
Run YOLOv8 Model
  ↓
Detect Objects
  ↓
Draw Bounding Boxes
  ↓
Display Output
  ↓
End
```

- **System Components**
  - **Backbone** → Feature extraction
  - **Neck** → Feature aggregation
  - **Head** → Detection output

- **Memory & Buffer Handling**
  - Batch processing for training
  - GPU memory optimization
  - Efficient inference pipeline

🔷 3.1 Code Submission (GitHub Link)

Smart City Traffic Forecasting Project Repository:
👉 https://github.com/Brahmavartika-108/smart-city-traffic-forecasting

Crop vs Weed Detection using YOLOv8 Repository:
👉 https://github.com/Brahmavartika-108/crop-vs-weed-detection-yolov8

---

4 🔷 3.2 Report Submission (GitHub Link)

Final Internship Report Repository:

👉 https://github.com/Brahmavartika-108/internship-report

4. PERFORMANCE TEST

This section evaluates how the developed systems perform under **real-world constraints**, ensuring their applicability in industrial environments. The performance testing focuses on identifying system limitations, validating efficiency, and analyzing robustness.

4.1 SMART CITY TRAFFIC FORECASTING SYSTEM

4.1.1 Identified Constraints

1. Accuracy Constraints

- Forecasting errors (MAE, RMSE)
- Difficulty in predicting:
  - Sudden spikes (accidents, events)
  - Irregular traffic patterns

---

2. Computational Constraints

- LSTM models require:
  - High computational power
  - GPU acceleration for training
- Increased training time compared to ARIMA/SARIMA

---

3. Real-Time Processing Constraints

- Need for:
  - Low-latency predictions
  - Fast API response

- **4. Data Dependency Constraints**
  - Model heavily depends on:
    - Quality of historical data
    - Availability of continuous time-series data

- **5. Scalability Constraints**
  - Expanding from:
    - Single junction → multiple city-wide nodes
  - Increased data volume affects performance

- ## 4.1.2 Handling of Constraints

Constraint	Solution Implemented
Accuracy	Used LSTM (captures non-linear patterns)
Seasonality	SARIMA introduced
Computation	Used optimized batch sizes & lightweight architecture
Real-time	Deployed via REST API
Scalability	Modular pipeline design

- ## 4.1.3 Test Plan / Test Cases
- **Test Plan Overview**

Test ID	Test Objective	Input	Expected Output
T1	Model Accuracy	Historical traffic data	Low MAE/RMSE
T2	Peak Hour Prediction	Peak hour data	Accurate spike prediction
T3	API Response Time	Real-time request	Fast response (<1 sec)
T4	Multi-Junction Handling	Multiple junction data	Stable predictions
T5	Missing Data Handling	Incomplete dataset	Reasonable interpolation

---

- ## 4.1.4 Test Procedure

1. Load dataset into system
2. Apply preprocessing and feature engineering
3. Train models (ARIMA, SARIMA, LSTM)
4. Generate predictions on test dataset
5. Compare:
    - Predicted vs actual values
6. Evaluate using:
    - MAE
    - RMSE
7. Deploy model via API
8. Send real-time requests and measure:
    - Response time
    - Stability

- ### 4.1.5 Performance Outcome
- **Accuracy Results**

- ARIMA:
    - MAE ≈ 34.17
    - RMSE ≈ 40.86
- SARIMA:
    - Improved accuracy (lower error)
- LSTM:
    - Best performance
    - Captured:
        - Peak traffic
        - Long-term dependencies

- **Speed Performance**

- API response time:
    - ~ milliseconds to <1 second
- Suitable for:
    - Real-time systems

- **Scalability Performance**

- Handles:
  - Multi-junction data
- Modular design allows:
  - Expansion to city-scale systems

- **Observed Limitations**

- Reduced accuracy during:
  - Sudden anomalies
- Dependency on:
  - Data quality

- **Recommendations**

- Integrate:
  - Real-time data feeds (IoT sensors)
- Use:
  - Hybrid models (LSTM + external factors)
- Deploy on:
  - Cloud infrastructure for scalability

4.2 CROP vs WEED DETECTION SYSTEM

4.2.1 Identified Constraints

1. Accuracy Constraints

- Misclassification due to:
  - Similar visual features
  - Dense vegetation

- **2. Computational Constraints**
  - YOLOv8 requires:
    - GPU for training
    - High memory usage

- **3. Inference Speed Constraints**
  - Real-time detection requires:
    - Low latency
    - Fast image processing

- **4. Environmental Constraints**
  - Performance affected by:
    - Lighting conditions
    - Shadows
    - Weather variations

- **5. Hardware Constraints**
  - Edge devices (Raspberry Pi):
    - Limited RAM
    - Limited processing power

- ◆ **4.2.2 Handling of Constraints**

Constraint	Solution Implemented
Accuracy	Data augmentation + tuning
Speed	YOLOv8n (lightweight model)
Computation	GPU training (Tesla T4)
Deployment	ONNX & TorchScript export
Environment	Augmentation for robustness

4.2.3 Test Plan / Test Cases

Test Plan Overview

Test ID	Test Objective	Input	Expected Output
T1	Detection Accuracy	Validation images	Correct bounding boxes
T2	Multi-object Detection	Dense images	Multiple detections
T3	Speed Test	Real-time input	Fast inference
T4	Lighting Variation	Low/high light images	Stable detection
T5	Edge Deployment	Low-power device	Acceptable performance

4.2.4 Test Procedure

1. Prepare dataset (train + validation)
2. Train YOLOv8 model
3. Run inference on validation set
4. Measure:
    - Precision
    - Recall
    - mAP
5. Test on:
    - Unseen real-world images
6. Measure:
    - Detection accuracy
    - Inference time
7. Deploy model (ONNX/TorchScript)
8. Simulate:
    - Edge and cloud environments

4.2.5 Performance Outcome

Accuracy Results

- Precision ≈ 0.82
- Recall ≈ 0.76
- mAP@50 ≈ 0.82

- mAP@50–95 ≈ 0.54

- **Speed Performance**

  - YOLOv8n:
    - Fast inference
    - Suitable for real-time detection

- **Robustness**

  - Works well in:
    - Normal lighting
    - Structured crop patterns
  - Slight drop in:
    - Low light
    - Highly cluttered backgrounds

- **Deployment Performance**

  - ONNX:
    - Cross-platform support
  - TorchScript:
    - Faster inference in PyTorch environments

- **Observed Limitations**

  - Sensitivity to:
    - Extreme lighting
  - Dataset size limitation (1300 images)

5 My learnings

The internship gave me hands-on experience across the full Data Science and Machine Learning lifecycle, helping me move from theory to practical application. I learned to build end-to-end ML pipelines, including data preprocessing, EDA, feature engineering, model development, evaluation, and deployment.

I gained strong expertise in time-series forecasting using ARIMA, SARIMA, LSTM, and GRU, along with a solid understanding of deep learning concepts like model tuning, overfitting control, and optimization. I also worked on computer vision tasks using YOLOv8, learning object detection techniques and evaluation metrics such as precision, recall, and mAP.

Beyond modeling, I developed skills in evaluating model performance, interpreting results, and improving systems. A key takeaway was learning how to deploy models as real-world applications using tools like Flask, FastAPI, Streamlit, and model export formats.

Practically, I learned to handle real-world challenges like limited resources, noisy data, and balancing accuracy with efficiency. I understood the importance of feature engineering and adopted an iterative approach to model building and problem-solving.

Professionally, I improved my project structuring, documentation, and industry-oriented thinking, focusing on scalability and real-world impact. Overall, the internship strengthened my foundation and prepared me for roles such as Data Scientist, Machine Learning Engineer, or AI Engineer, marking my transition from a student to an industry-ready professional.

6 Future work scope

Although significant progress was achieved, there are several enhancements that can further improve the system.

- ## 5.1 Smart City Traffic Forecasting
- **1. Integration with Real-Time Data**

  - Use live traffic APIs or IoT sensors
  - Improve prediction accuracy in real-world scenarios

- **2. Advanced Models**
  - Transformer-based models (e.g., Time-Series Transformers)
  - Hybrid models (LSTM + external features like weather)

- **3. City-Wide Scaling**

  - Extend system to:
    - Multiple cities
    - Large-scale traffic networks

- **4. Mobile/Web Application**

  - Develop user-facing apps for:
    - Traffic monitoring
    - Route optimization

- **5. Cloud Deployment**

  - Deploy on:

- AWS / Google Cloud
- Enable scalability and high availability

- ## 🔹 5.2 Crop vs Weed Detection
- ### 1. Larger and Diverse Dataset

- Increase dataset size
- Include:
  - Different crops
  - Different geographical conditions

- ### 2. Drone Integration

- Use aerial imagery for:
  - Large-scale field monitoring

- ### 3. Real-Time Edge Deployment

- Optimize model for:
  - Raspberry Pi
  - Jetson Nano

- ### 4. Automated Action Systems

- Integrate with:
  - Robotic weed removal systems
  - Smart sprayers

- ### 5. Multi-Class Detection

- Extend from:
  - Crop vs Weed → Multiple plant categories

5.3 Overall System Enhancements

- Build unified AI platform combining:
  - Smart city systems
  - Smart agriculture systems
- Improve:
  - Robustness
  - Generalization
- Focus on:
  - Sustainability and efficiency